

Unit 4 Finding Optimal Paths

In Artificial Intelligence (AI), finding optimal paths involves using algorithms like A* and Dijkstra's to efficiently navigate through a graph or grid, finding the shortest or most efficient path between two points.

Key Concepts and Algorithms:

- **Pathfinding:**

The general problem of finding a route or path between two points, often with constraints or costs associated with each path.

- **Graphs and Grids:**

AI pathfinding problems are often represented as graphs (nodes and edges) or grids, where nodes represent locations and edges represent connections or paths between them.

- **Algorithms:**

- **A Algorithm:*** A popular and efficient algorithm that uses a heuristic function to guide the search towards the goal, balancing the cost of the path taken with the estimated cost to reach the goal.
- **Dijkstra's Algorithm:** Another algorithm used to find the shortest path in a graph, but it doesn't use a heuristic function, making it slower for large graphs compared to A*.
- **Breadth-First Search (BFS):** A simple algorithm that explores all paths at a given depth before moving to the next depth, useful for finding the shortest path in an unweighted graph.
- **Depth-First Search (DFS):** Explores as far as possible along each branch before backtracking, useful for finding all possible paths, but not necessarily the shortest.

- **Heuristics:**

A heuristic function estimates the cost of reaching the goal from a given node, helping A* to prioritize promising paths.

- **Optimization:**

Finding the best possible path, often meaning the shortest path in terms of distance or cost, but can also involve other criteria.

4.1. Brute Force

In AI, a brute-force search algorithm, also known as exhaustive or complete search, systematically checks every possible solution until the optimal one is found, though it can be computationally expensive.

- **What it is:**

Brute-force search is a general problem-solving technique and algorithmic paradigm that involves generating a list of all the possible candidates for a solution and then testing the validity of every single candidate. It is also known as exhaustive search or generate and test.

Brute-force search is a straightforward problem-solving technique where you consider every possible solution or combination to find the one that meets the criteria.

- **How it works:**

The algorithm generates a list of all potential solutions and then tests each one until the correct or optimal one is identified.

- **Examples:**
 - **Password Cracking:** Trying every possible combination of characters to crack a password.
 - **Exhaustive Search:** Solving problems by considering every single possibility.
 - **Sorting Algorithms:** Some sorting algorithms like bubble sort can be considered brute-force approaches.
 - **Linear Search:** A simple search algorithm that examines each element in a dataset until a match is found.
- **Advantages:**
 - **Guaranteed Solution:** If a solution exists, a brute-force search is guaranteed to find it.
 - **Simplicity:** The algorithm is easy to understand and implement.
- **Disadvantages:**
 - **Computational Cost:** Brute-force algorithms can be very time-consuming and resource-intensive, especially for large datasets or complex problems.
 - **Scalability Issues:** They often struggle to handle larger problem instances as the input size grows exponentially.
- **When to use:**
 - When other, more efficient algorithms are not available or practical.
 - When the problem space is relatively small, and computational resources are not a major concern.
- **Alternatives:**
 - **Heuristic Search:** Algorithms that use "rules of thumb" or educated guesses to find solutions more quickly.
 - **Greedy Algorithms:** Algorithms that make locally optimal choices at each step to reach a global solution.
 - **Divide and Conquer:** Algorithms that break down a problem into smaller subproblems that can be solved independently.

What is Brute-force search?

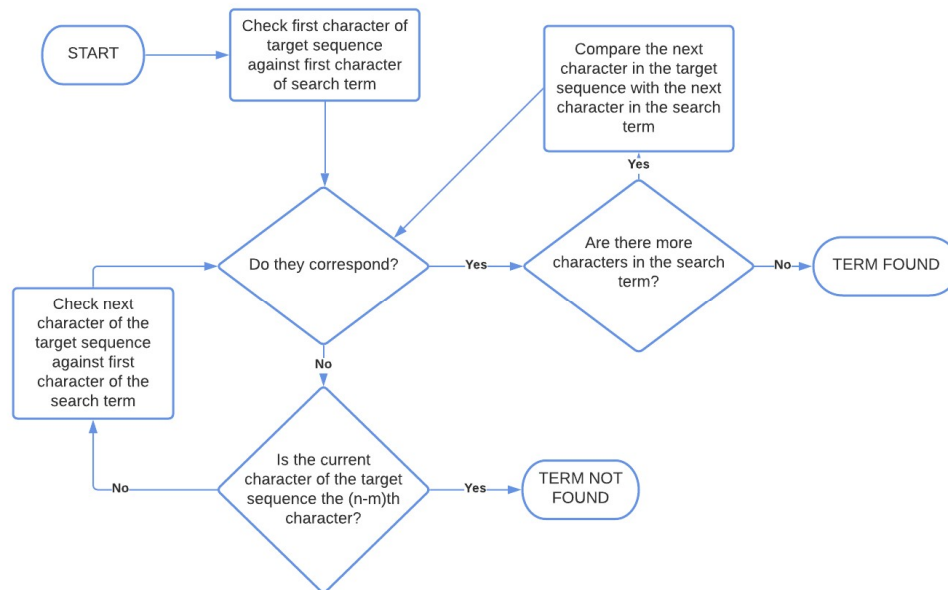
It involves systematically enumerating all possible candidates for the solution and checking whether each candidate satisfies the problem's statement. Brute-force search is the most common search algorithm because it does not need any domain knowledge. However, it can take a lot of run time and could be rather inefficient.

Essentially, in brute-force search, you have to iterate through all the candidates, checking whether they satisfy the problem's statement. You check each candidate till you find the right one. So, if the candidate that satisfies the problem statement is at the end of the collection, you'll have to check all the candidates in the collection. It's even possible that the candidate may not exist in the collection, in which case you'd still end up checking every single candidate in the collection, but to no avail.

If the solution exists in the collection, you will be able to find it with brute-force search, however, the costs involved in this technique are proportional to the number of candidate solutions.

In the real world, for a lot of problems, the number of candidates is far too high (Combinatorial explosion). Because of this phenomenon, the brute-force method is applied when problem-

specific heuristics can be applied to reduce the number of candidate solutions available to a size that is manageable.



What are brute-force algorithms?

Brute-force algorithms rely purely on sheer computing power to find solutions for problems. They try every single solution that could possibly solve the problem instead of employing advanced techniques to get the job done in a more efficient manner.

Some of the features of brute force algorithms are:

- A brute force algorithm is an intuitive, direct, and straightforward technique of problem solving that enumerates all the possible ways or all the possible solutions to a specified problem.
- Several problems that arise in day-to-day life are solved through the use of the brute force strategy, for example exploring all the paths to a nearby park to find the minimum shortest path, or arranging the books in a rack using all the possibilities to optimize the rack spaces, etc.
- Even though optimal algorithms are also possible, a lot of daily life activities use a brute force nature.

How can you speed up brute-force searches?

One of the ways in which you can speed up a brute-force algorithm is by reducing the search space (reducing the set of candidate solutions) by making use of heuristics that are specific to the problem class.

As an example, in the eight queens problem the challenge, the challenge involves placing eight queens on a standard chessboard so that no queen attacks any other queen. Due to the fact that each of the queens can be placed in any of the 64 squares, in principle there are $64^8 = 281,474,976,710,656$ possibilities to consider. However, since all the queens are alike, and no two queens can be placed on the same square, the candidates are all possible ways of choosing of a set of 8 squares from the set of 64 squares. Which means $64 \text{ choose } 8 = \frac{64!}{(56! \cdot 8!)} = 4,426,165,368$ candidate solutions – about

1/60,000 of the previous estimate. In addition to this, no arrangement that features two queens on the same row or the same column can be an acceptable solution. Because of this, you can further restrict the set of candidates to those arrangements.

This example demonstrates that a small amount of analysis would quite often lead to very significant reductions in the number of candidate solutions, and could potentially turn an intractable problem into a trivial one.

In some situations, the analysis could reduce the candidates to the set of all valid solutions. This means that it could yield an algorithm that directly enumerates all the desired solutions (or finds one solution, as appropriate), without wasting time on tests and the generation of invalid candidates. As an example, for the problem "find all integers between 1 and 1,000,000 that are evenly divisible by 417" a naive brute-force solution would generate all integers in the range and test every single one of them for divisibility. However, that problem can be solved with a far greater degree of efficiency by starting with 417 and repeatedly adding 417 until the number exceeds 1,000,000 – which takes only 2398 ($= 1,000,000 \div 417$) steps, and does not require any tests.

What are the advantages of the brute-force search?

The advantages of the brute-force search are:

- If all the possible solutions are listed down, then you can be guaranteed that the brute-force approach will find the correct solution.
- It is applicable to problems that exist in a very wide range of domains.
- It is useful for solving small and simple problems.
- It is very simple, does not require any domain knowledge, and can be used as a [comparison benchmark](#).

What are the disadvantages of the brute-force search?

Here are the disadvantages of brute-force search:

- The brute-force method is very inefficient. For real-time problems, algorithm analysis quite frequently goes above the $O(N!)$ order of growth.
- Finding the right solution by this method consumes a lot of time. These algorithms run rather slowly.
- Instead of using a good algorithm design, these algorithms just rely on compromising the power of the computer.
- These algorithms are neither constructive nor creative in comparison to algorithms that are constructed through the use of some other design paradigms.

4.2 Branch and Bound

Branch and bound is an AI search algorithm that systematically explores a solution space by branching into subproblems and using bounds to prune branches that cannot lead to an optimal solution, effectively reducing the search space.

Here's a more detailed explanation:

- **Problem Decomposition:**

Branch and bound tackles optimization problems by breaking them down into smaller, manageable subproblems (branches).

- **Bounding:**

Each branch is evaluated using upper and lower bounds to determine its potential to yield an optimal solution.

- **Pruning:**

Branches that cannot outperform the current best solution are discarded, significantly reducing the search space and improving efficiency.

- **Systematic Exploration:**

The algorithm explores branches systematically, ensuring that all potential solutions are considered or pruned.

- **Applications:**

Branch and bound is commonly used in problems like the traveling salesman problem, job scheduling, and other combinatorial optimization tasks.

- **Relationship to other algorithms:**

Branch and bound can be seen as a generalization of other search algorithms like A* and AO*, which are special cases of the general branch and bound formulation.

In essence, branch and bound is a powerful technique for finding optimal solutions in complex search spaces by strategically exploring and discarding branches based on bounds, leading to efficient and effective problem-solving.

Concept:

Step 1: Traverse the root node.

Step 2: Traverse any neighbour of the root node that is maintaining least distance from the root node.

Step 3: Traverse any neighbour of the neighbour of the root node that is maintaining least distance from the root node.

Step 4: This process will continue until we are getting the goal node.

Algorithm:

Step 1: PUSH the root node into the stack.

Step 2: If stack is empty, then stop and return failure.

Step 3: If the top node of the stack is a goal node, then stop and return success.

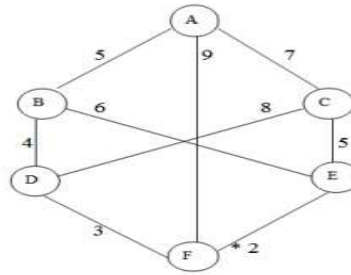
Step 4: Else POP the node from the stack. Process it and find all its successors. Find out the path containing all its successors as well as predecessors and then PUSH the successors which are belonging to the minimum or shortest path.

Step 5: Go to step 5.

Step 6: Exit.

Implementation:

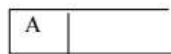
Let us take the following example for implementing the Branch and Bound algorithm.



Figure

Step 1:

Consider the node A as our root node. Find its successors i.e. B, C, F. Calculate the distance from the root and PUSH them according to least distance.



B: $0+5 = 5$ (The cost of A is 0 as it is the starting node)

F: $0+9 = 9$

C: $0+7 = 7$

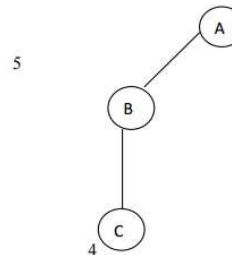
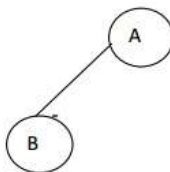
Here B (5) is the least distance.

As B is on the top of the stack so calculate the neighbours of B.

D: $0+5+4 = 9$

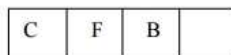
E: $0+5+6 = 11$

The least distance is D from B. So it will be on the top of the stack.



Step 2:

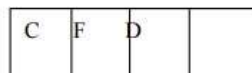
Now the stack will be



A

Step 3:

As the top of the stack is D. So calculate neighbours of D.



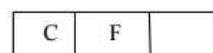
B

C: $0+5+4+8 = 17$

F: $0+5+4+3 = 12$

The least distance is F from D and it is our goal node. So stop and return success.

Step 4:



D

Hence the searching path will be A-B -D-F

Advantages:

As it finds the minimum path instead of finding the minimum successor so there should not be any repetition.

The time complexity is less compared to other algorithms.

Disadvantages:

The load balancing aspects for Branch and Bound algorithm make it parallelization difficult.

The Branch and Bound algorithm is limited to small size network. In the problem of large networks, where the solution search space grows exponentially with the scale of the network, the approach becomes relatively prohibitive.

4.3 Refinement Search

In AI, "refinement search" or "query refinement" refers to the process of iteratively improving search results or AI responses by adjusting the initial query or prompt to achieve more accurate and relevant outcomes.

Here's a breakdown of the concept:

- **Why Refine?**

Initial search queries or AI prompts might be too broad or ambiguous, leading to irrelevant or incomplete results. Refinement allows users to narrow down the search scope and get closer to their desired information.

- **How it works:**

- **User Interaction:** Users can refine their search by adding keywords, filters, or rephrasing their query based on the initial results.
- **AI Assistance:** AI systems can also suggest refinements or automatically adjust the search based on user feedback or the context of the query.
- **Examples:**
 - **E-commerce:** A user searching for "hats" might refine the search by selecting "size," "style," or "color".
 - **AI Chatbots:** A user asking "what's the weather?" might refine their query by adding "in Hyderabad" or "today".
 - **Patent Search:** Refinements can help users uncover more prior art by providing additional information to the AI.

- **Benefits:**

- **Improved Accuracy:** Refinement leads to more precise and relevant results.
- **Enhanced User Experience:** Users can quickly find what they need without sifting through irrelevant information.
- **Better AI Performance:** Refinement helps AI systems understand user needs better and provide more tailored responses.

- **Refinement Strategies:**

- **Adding Keywords:** Including more specific terms in the query.
- **Using Filters:** Applying criteria like date, location, or type to narrow the results.
- **Rephrasing:** Changing the wording of the query to better reflect the user's intent.
- **Using Search Operators:** Employing operators like AND, OR, and NOT to refine the search.

4.4 Dijkstra's Algorithm:

- Dijkstra's algorithm is a classic algorithm used to find the shortest paths from a single source node to all other nodes in a graph.
- It's a greedy algorithm, meaning it makes locally optimal choices at each step to arrive at a globally optimal solution (the shortest paths).
- It works by iteratively exploring the graph, marking nodes as visited and updating the shortest known distances to them.

How it works:**1. 1. Initialization:**

- Assign a distance of 0 to the starting node and infinity to all other nodes.
- Create a set of unvisited nodes.

2. 2. Iteration:

- While there are unvisited nodes:
 - Select the unvisited node with the smallest known distance.
 - Mark the selected node as visited.
 - For each neighbor of the selected node:
 - Calculate the distance from the starting node to the neighbor through the selected node.
 - If this distance is shorter than the current known distance to the neighbor, update the neighbor's distance.

3. 3. Termination:

- The algorithm terminates when all nodes have been visited or when the destination node is reached.

AI Applications:

- **Pathfinding:**

Dijkstra's algorithm is widely used in AI for pathfinding problems, such as finding the shortest route between two points in a map or a game world.

- **Robotics:**

It can be used for robot navigation, helping robots plan their paths to reach specific locations.

- **Network Routing:**

In computer networks, Dijkstra's algorithm can be used to find the shortest paths for data packets to travel between routers.

- **Logistics and Transportation:**

It can be used to optimize delivery routes, traffic flow, and other logistical tasks.

Limitations:

- **Non-negative edge weights:**

Dijkstra's algorithm only works correctly for graphs with non-negative edge weights.

- **Single-source:**

It finds shortest paths from a single starting node to all other nodes, not between all pairs of nodes.

- **Static graphs:**

It's best suited for static graphs where the edge weights don't change during the algorithm's execution.

Principle of Dijkstra's Algorithm

To find the shortest path between two given vertices of a graph, we will follow the following mentioned steps of the algorithm/approach, which are:

if $\text{dist}(u) + \text{len}(u,v) < \text{dist}(v)$

$\text{dist}(v) = \text{dist}(u) + \text{len}(u,v)$

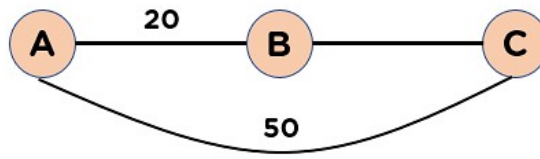
Where,

$\text{dist}(u)$ = Source Node

$\text{dist}(v)$ = Destination Node

Note: By default, the source node's immediate and non-immediate distance to the other nodes in the graph is " ∞ (Infinite)."

For Example: Find the shortest path for the given graph.



1. We will find the shortest path from node A to the other nodes in the graph, assuming that node A is the source.

2. For node B, apply the above-discussed [algorithm](#), then on applying values:

if $0 + 20 < \infty$

>[TRUE]

Node A to Node B = 20

3. For node C, on applying the approach from node A, we have:

if $0 + 50 < \infty$

>[TRUE]

Node A to Node C = 50

4. For node C, from node B, on applying the algorithm, we have:

if $20 + 10 < \infty$

>[TRUE]

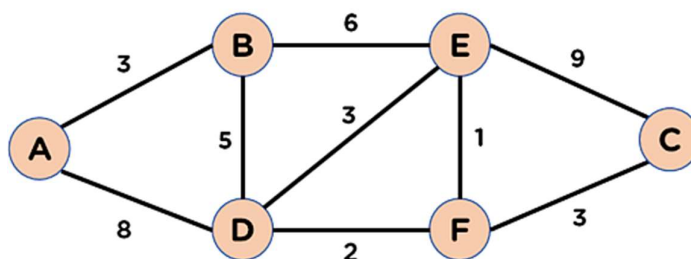
Node B to Node C = 30

5. By the value obtained from step 3, we change the shortest distance between node A to Node C to 30 from the previous distance of 50.

Now that we are clear with the approach behind the algorithm let's apply it for implementation.

Implementing Dijkstra's Algorithm:

Let's apply Dijkstra's Algorithm for the graph given below, and find the shortest path from node A to node C:



Solution:

1. All the distances from node A to the rest of the nodes is ∞ .
2. Calculating the distance between node A and the immediate nodes (node B & node D):
For node B,
Node A to Node B = 3
For node D,
Node A to Node D = 8
3. Choose the node with the shortest distance to be the current node from unvisited nodes, i.e., node B. Calculating the distance between node B and the immediate nodes:
For node E,
Node B to Node D = $3+5 = 8$
For node E,
Node B to Node E = $3+6 = 9$
4. Choose the node with the shortest distance to be the current node from unvisited nodes, i.e., node D. Calculating the distance between node D and the immediate nodes:
For node E,
Node D to Node E = $8+3 = 11$ ($[9 < 11] > \text{TRUE}$: So, No Change)
For node F,
Node D to Node F = $8+2 = 10$
5. Choose the node with the shortest distance to be the current node from unvisited nodes, i.e., node E. Calculating the distance between node E and the immediate nodes:
For node C,
Node E to Node C = $9+9 = 18$
For node F,
Node E to Node F = $9+1 = 10$
6. Choose the node with the shortest distance to be the current node from unvisited nodes, i.e., node F. Calculating the distance between node F and the immediate nodes:
For node C,
Node F to Node C = $10+3 = 13$ ($[18 < 13]$ FALSE: So, Change the previous value)
So, after performing all the steps, we have the shortest path from node A to node C, i.e., a value of 13 units.

4.5 A* Algorithm

A* Search is an informed best-first search algorithm that efficiently determines the lowest cost path between any two nodes in a directed weighted graph with non-negative edge weights. This algorithm is a variant of Dijkstra's algorithm. A slight difference arises from the fact that an evaluation function is used to determine which node to explore next.

Evaluation Function

The evaluation function, $f(x)$, for the A* search algorithm is the following:

$$f(x) = g(x) + h(x)$$

Where $g(x)$ represents the cost to get to node x and $h(x)$ represents the estimated cost to arrive at the goal node from node x .

For the algorithm to generate the correct result, the evaluation function must be admissible, meaning that it never overestimates the cost to arrive at the goal node.

The Algorithm

The A* algorithm is implemented in a similar way to Dijkstra's algorithm. Given a weighted graph with non-negative edge weights, to find the lowest-cost path from a start node **S** to a goal node **G**, two lists are used:

- An **open list**, implemented as a priority queue, which stores the next nodes to be explored. Because this is a priority queue, the most promising candidate node (the one with the lowest value from the evaluation function) is always at the top. Initially, the only node in this list is the start node **S**.
- A **closed list** which stores the nodes that have already been evaluated. When a node is in the closed list, it means that the lowest-cost path to that node has been found.

To find the lowest cost path, a search tree is constructed in the following way:

1. Initialize a tree with the root node being the start node **S**.
2. Remove the top node from the open list for exploration.
3. Add the current node to the closed list.
4. Add all nodes that have an incoming edge from the current node as child nodes in the tree.
5. Update the lowest cost to reach the child node.
6. Compute the evaluation function for every child node and add them to the open list.

Just like in Dijkstra's algorithm, the lowest cost is updated as the algorithm progresses in the following way:

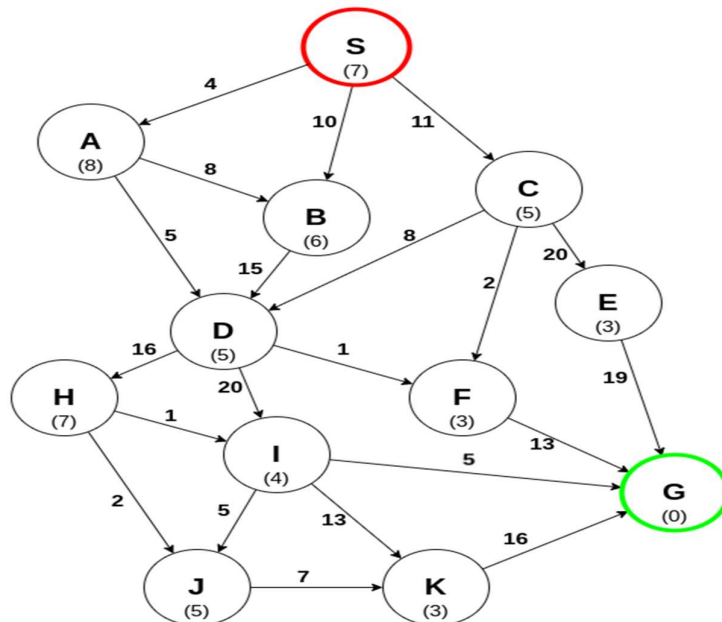
$$\text{current_lowest_cost} = \min(\text{current_lowest_cost}, \text{parent_node_cost} + \text{edge_weight})$$

All nodes except for the start node start with a lowest cost of infinity. The start node has an initial lowest cost of 0. The algorithm concludes when the goal node **G** is removed from the open list and explored, indicating that a shortest path has been found. The shortest path is the path from the start node **S** to the goal node **G** in the tree.

Note: The algorithm ends when the goal node **G** has been explored, NOT when it is added to the open list.

Example

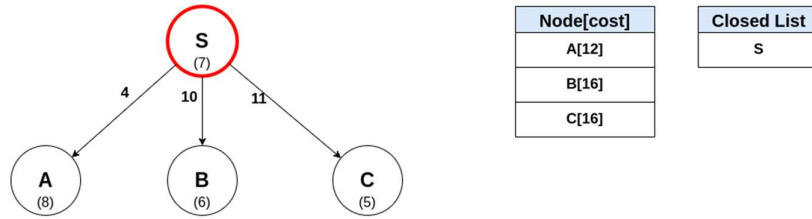
Consider the following example of trying to find the shortest path from **S** to **G** in the following graph:



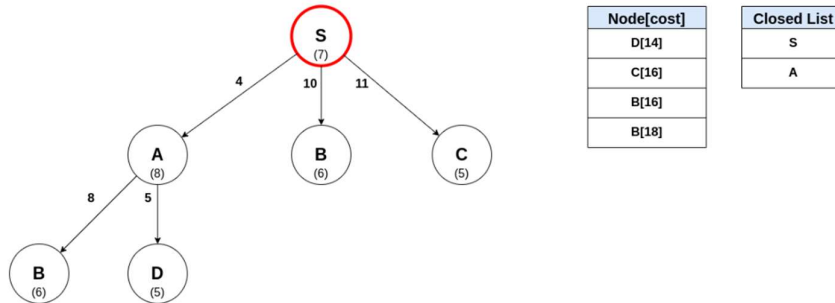
Each edge has an associated weight, and each node has a heuristic cost (in parentheses).

An open list is maintained in which the node S is the only node in the list. The search tree can now be constructed.

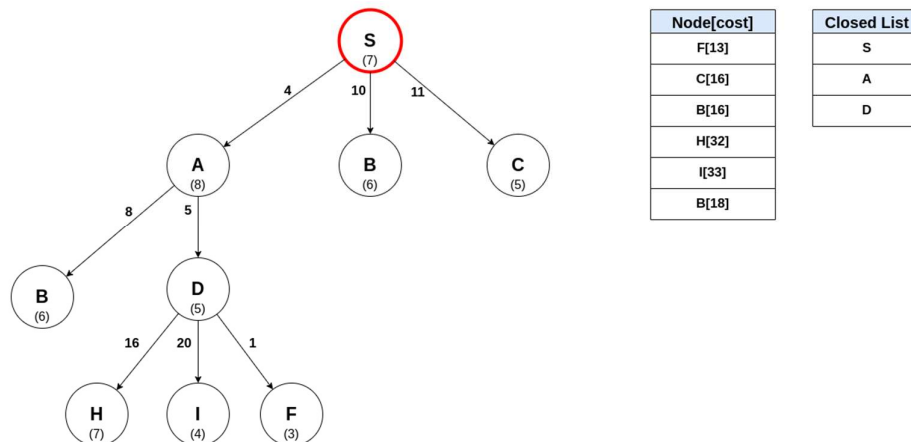
Exploring S:



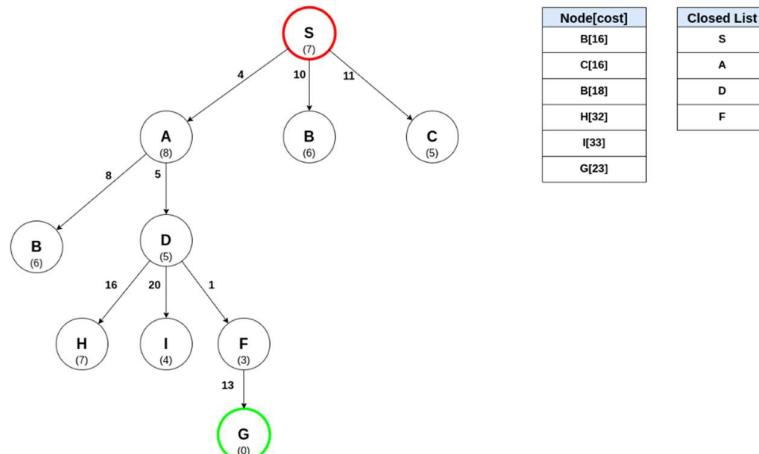
A is the current most promising path, so it is explored next:



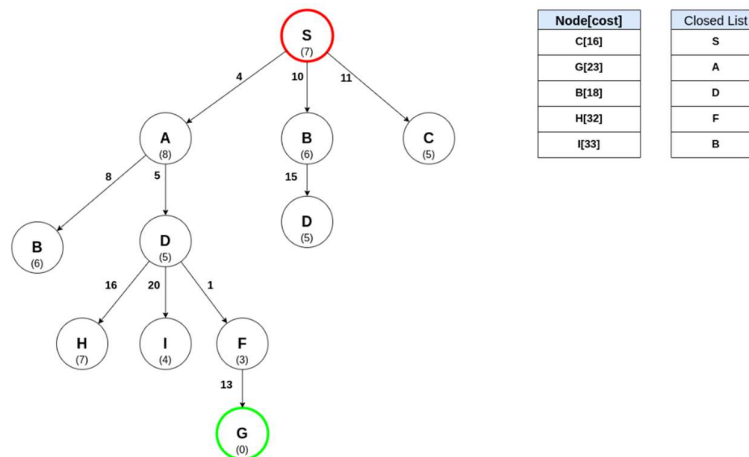
Exploring D:



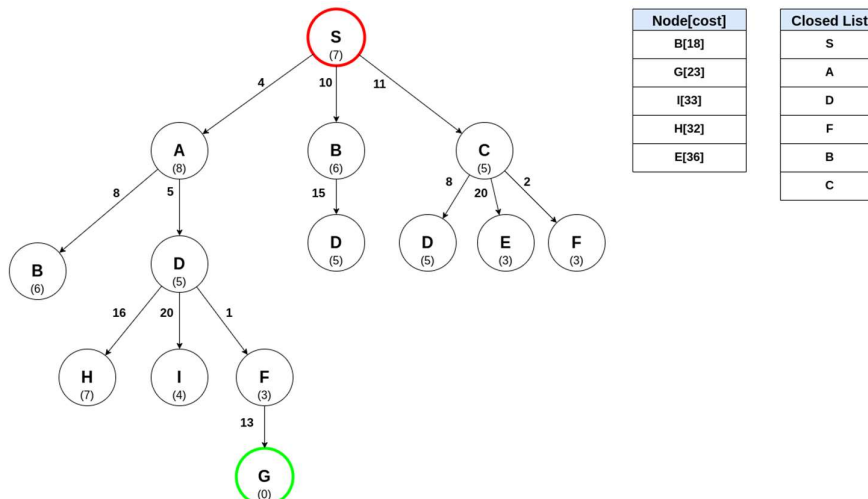
Exploring F:



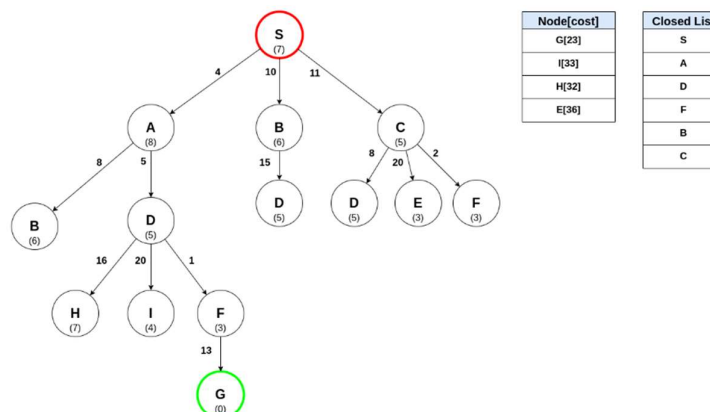
Notice that the goal node **G** has been found. However, it hasn't been explored, so the algorithm continues because there may be a shorter path to G. The node **B** has two entries in the open list: one at a cost of 16 (child of **S**) and one at a cost of 18 (child of **A**). The one with the lowest cost is explored next:



Exploring C:



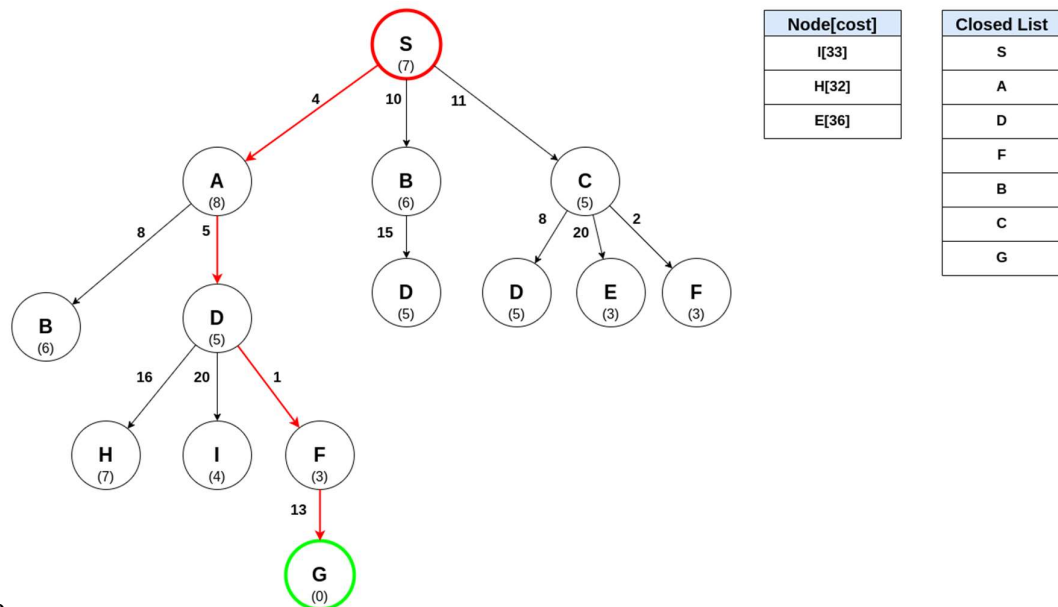
The next node in the open list is again **B**. However, because **B** has already been explored, meaning the shortest path to **B** has been found, it is not explored again, and the algorithm continues to the next candidate.



The next node to be explored is the goal node **G**, meaning the shortest path to **G** has been found! The path is constructed by tracing the graph backwards from **G** to **S**:

Using the A* Algorithm

This algorithm is guaranteed to find the shortest path if one exists. One of the main uses of this algorithm is route planning, but there are many other uses.

**Example****Code**

Here is an example of the A* algorithm implemented in Python that solves the above example graph:
 from heapq import heappop, heappush

```
def a_star_search(graph: dict, start: str, goal: str, heuristic_values: dict) -> int:
```

```
    """ A* search algorithm implementation.
```

```
    @param graph: The graph to search.
```

```
    @param start: The starting node.
```

```
    @param goal: The goal node.
```

```
    @param heuristic_values: The heuristic values for each node. The goal node must be admissible,
    and the heuristic value must be 0.
```

```
    @return: The path cost from the start node to the goal node.
```

```
    """
```

```
    # A min heap is used to implement the priority queue for the open list.
```

```
    # The heapq module from Python's standard library is utilized.
```

```
    # Entries in the heap are tuples of the form (cost, node), ensuring that the entry with the lowest
    cost is always smaller during comparisons.
```

```
    # The heapify operation is not required, as the heapq module maintains the heap invariant after
    every push and pop operation.
```

```
    # The closed list is implemented as a set for efficient membership checking.
```

```
    open_list, closed_list = [(heuristic_values[start], start)], set()
```

```
    while open_list:
```

```
        cost, node = heappop(open_list)
```

The algorithm ends when the goal node has been explored, NOT when it is added to the open list.

```
if node == goal:
    return cost
```

```
if node in closed_list:
    continue
```

```
closed_list.add(node)
```

Subtract the heuristic value as it was overcounted.

```
cost -= heuristic_values[node]
```

```
for neighbor, edge_cost in graph[node]:
    if neighbor in closed_list:
        continue
```

$f(x) = g(x) + h(x)$, where $g(x)$ is the path cost and $h(x)$ is the heuristic.

```
neighbor_cost = cost + edge_cost + heuristic_values[neighbor]
```

```
heappush(open_list, (neighbor_cost, neighbor))
```

```
return -1 # No path found
```

```
EXAMPLE_GRAPH = {
    'S': [('A', 4), ('B', 10), ('C', 11)],
    'A': [('B', 8), ('D', 5)],
    'B': [('D', 15)],
    'C': [('D', 8), ('E', 20), ('F', 2)],
    'D': [('F', 1), ('I', 20), ('H', 16)],
    'E': [('G', 19)],
    'F': [('G', 13)],
    'H': [('J', 2), ('I', 1)],
    'I': [('K', 13), ('G', 5), ('J', 5)],
    'J': [('K', 7)],
    'K': [('G', 16)]
}
```

Node heuristic values (admissible heuristic values for the nodes)

```
EXAMPLE_HEURISTIC_VALUES = {
    'S': 7,
    'A': 8,
    'B': 6,
    'C': 5,
    'D': 5,
    'E': 3,
```

```
'F': 3,  
'G': 0,  
'H': 7,  
'I': 4,  
'J': 5,  
'K': 3  
}
```

```
EXAMPLE_RESULT = a_star_search(EXAMPLE_GRAPH, 'S', 'G',  
EXAMPLE_HEURISTIC_VALUES)  
print(EXAMPLE_RESULT)
```

The code above produces the following output: 23

Complexity Analysis

For time complexity, one might notice that each heappush corresponds to an edge, which would be the dominating complexity for most cases. Indeed, A* is equivalent to Dijkstra's algorithm when the heuristic function is 0, and A* is equivalent to Dijkstra's algorithm with reduced cost when the heuristic function is admissible i.e. $O(V+E\log(V))$ time complexity.

However, a good heuristic function can drastically decrease A*'s complexity. The idea here is we need to look at exponentially fewer nodes with a better heuristic. So the time and space complexity are actually $O(b^d)$, where b is the effective branching factor, i.e., an empirical average of neighboring nodes not in the closed list, and d is the search depth, i.e., optimal path length. The large space complexity is the biggest disadvantage of the A* search, giving rise to other algorithm variations.

4.6 Admissibility of A* Algorithm:

In the context of AI algorithms like A*, an admissible heuristic never overestimates the cost to reach the goal, ensuring that the algorithm finds the optimal path.

Here's a breakdown with an example:

- **What is Admissibility?**

In A*, a heuristic function $h(n)$ estimates the cost from node n to the goal. An admissible heuristic means $h(n)$ is always less than or equal to the actual cost to reach the goal from n , i.e., $h(n) \leq h^*(n)$.

- **Why is it Important?**

If the heuristic overestimates, A* might prematurely expand nodes that are not on the optimal path, leading to a suboptimal solution.

- **Example:**

Imagine a pathfinding problem where you want to travel from city A to city Z.

- A heuristic could be the straight-line distance between cities.
- If the heuristic estimates the distance from A to Z as 100km, and the actual shortest path is 90km, the heuristic is admissible because it underestimates the actual cost.
- However, if the heuristic estimates the distance as 120km, and the actual shortest path is 90km, the heuristic is inadmissible because it overestimates the actual cost.

- **In summary:**

Admissibility guarantees that A* will find the shortest path when using an admissible heuristic.

4.7 Iterative Deepening A* (IDA*) Algorithm:

IDA* is a variant of [depth-first search \(DFS\)](#) that iteratively deepens its search by incrementing the cost threshold, which controls the depth of the exploration. Unlike [A*](#), which explores all possible nodes within a threshold, IDA* uses a heuristic function to evaluate and prioritize the most promising nodes. This allows it to prune less promising paths, reducing memory usage while ensuring that the search focuses on optimal routes.

Key Features of IDA* Algorithm

1. **Graph Traversal Algorithm:** IDA* explores a graph by progressively deepening the search depth.
2. **Shortest Pathfinding:** It efficiently finds the **shortest path in a weighted graph** using a combination of DFS and A* techniques.
3. **Admissible Heuristic:** The heuristic function in IDA* ensures that the estimated cost never exceeds the actual cost, making it an **admissible heuristic**.
4. **Memory Efficiency:** As a **depth-first search algorithm**, IDA* uses far less memory compared to the traditional A* algorithm.
5. **Focused Search:** The algorithm is designed to explore **only the most promising nodes**, ensuring it doesn't go to unnecessary depths.

How the IDA* Algorithm Works?

The IDA* algorithm works by incrementally increasing the **threshold** based on the f-score of each node, which is calculated using the formula:

$$f(n)=g(n)+h(n)$$

$$f(n)=g(n)+h(n)$$

$$f(n)=\text{Actualcost}+\text{Estimated cost}$$

$$f(n)=\text{Actualcost}+\text{Estimated cost}$$

Where **h** is admissible.

Here,

- $f(n)$ = Total cost evaluation function.
- $g(n)$ = The actual cost from the initial node to the current node.
- $h(n)$ = Heuristic estimated cost from the current node to the goal state. it is based on the approximation according to the problem characteristics.

What is the F-score?

In the IDA* algorithm, F-score is a **heuristic function** that is used to estimate the cost of reaching the goal state from a given state. It is a combination of two other heuristic functions, **$g(n)$** and **$h(n)$** . It is used to determine the order in which the algorithm expands nodes in the search tree and thus, it plays an important role in how quickly the algorithm finds a solution.

A lower F-score indicates that a node is closer to the goal state and will be expanded before a node with a higher F-score. Simply it is nothing but **$g(n) + h(n)$** .

Step-by-Step Process of the IDA* Algorithm

1. **Initialization:** Set the root node as the current node and compute its f-score.
2. **Set Threshold:** Initialize a threshold based on the f-score of the starting node.
3. **Node Expansion:** Expand the current node's children and calculate their f-scores.

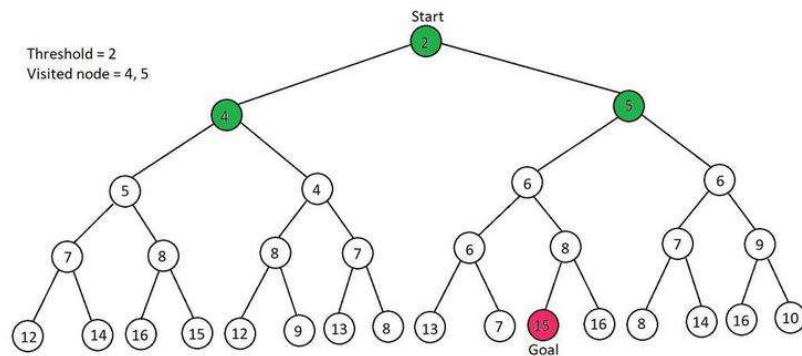
4. **Pruning:** If the f-score exceeds the threshold, prune the node and store it for future exploration.
5. **Path Return:** Once the goal node is found, return the path from the start node to the goal.
6. **Update Threshold:** If the goal is not found, increase the threshold based on the minimum pruned value and repeat the process.

Example of IDA* Algorithm

In the below tree, the **f score** is written inside the nodes means the f score is already computed and the start node is **2** whereas the goal node is **15**. the explored node is colored green color.

So now we have to go to a given goal by using IDA* algorithm.

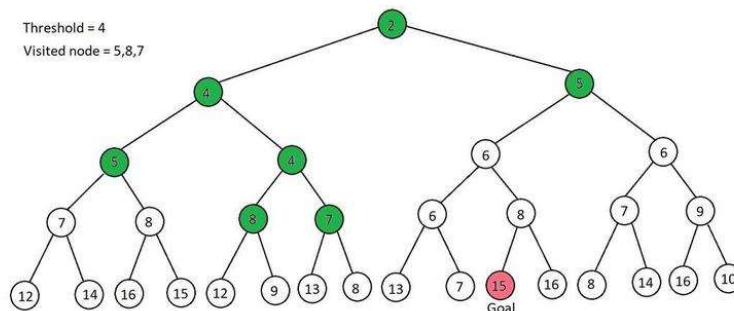
Iteration 1



Iteration 1

- Root node as current node i.e 2
- Threshold = current node value (2=2). So explore its children.
- 4 > Threshold & 5 > Threshold. So, this iteration is over and the pruned values are 4, and 5.

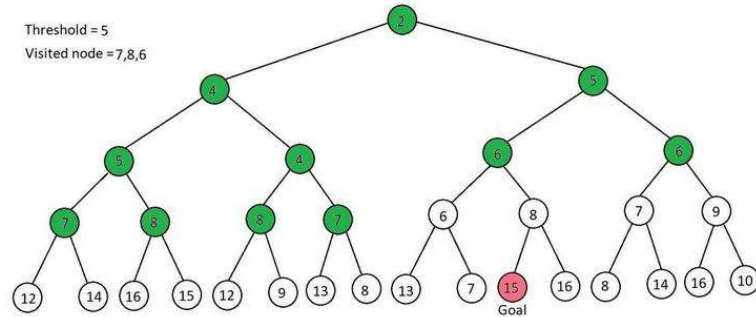
Iteration 2



Iteration 2

- In pruned values, the least is 4, So threshold = 4
- current node = 2 and 2 < threshold, So explore its children. i.e two children explore one by one
- So, first children 4, So, set current node = 4 i.e equal to the threshold, so, explored its children also i.e 5, 4 having 5 > threshold so, pruned it and explore second child of node 4 i.e 4, so set current node = 4 = threshold, and explore its children i.e 8 & 7 having both 8 & 7 > threshold so, pruned it. At the end of this, our pruned value is 5, 8, 7
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e 5 > threshold, So pruned it.
- So, our pruned value is 5, 8, 7.

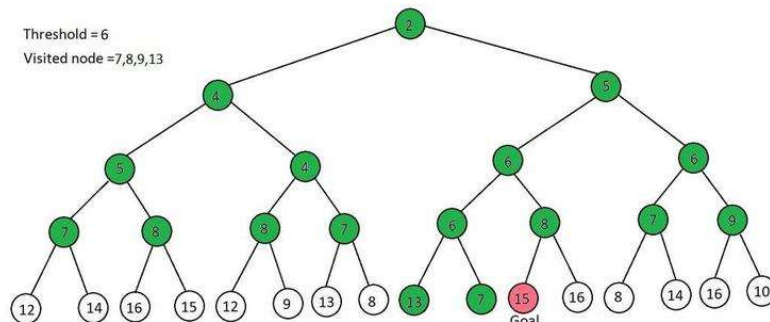
Iteration 3



Iteration 3

- In pruned values, the least is 5, So threshold = 5
- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, first children 4, So, set current node = $4 < \text{threshold}$, so, explored its children also i.e 5, 4 having $5 = \text{threshold}$ so explore its child also 7 & 8 $> \text{threshold}$. So, pruned it and explore the second child of node 4 i.e 4, so set current node = $4 < \text{threshold}$, and explore its children i.e 8 & 7 here, both 8 & 7 $> \text{threshold}$ so, pruned it. At the end of this, our pruned value is 7 & 8
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 = \text{threshold}$, so, explored its children also i.e 6 & 6, i.e both 6 & 6 $> \text{threshold}$. So pruned it
- So, our pruned value is 7,8 & 6

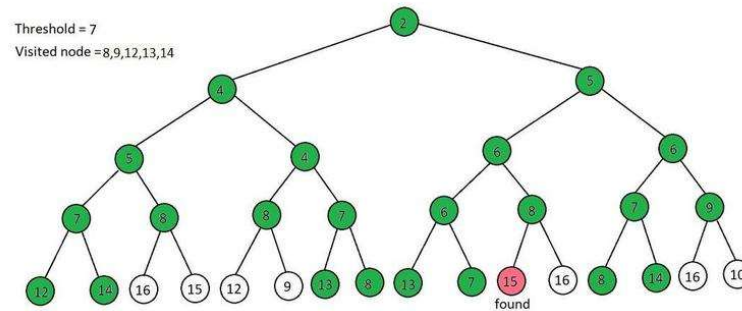
Iteration 4



- In pruned values, the least value is 6, So threshold = 6
- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, the first child is 4, So, set current node = $4 < \text{threshold}$, so, explored its children also i.e 5, 4 having $5 < \text{threshold}$ so explore its child also 7 & 8 $> \text{threshold}$. So, pruned it and explore the second child of node 4 i.e 4, so set current node = $4 < \text{threshold}$, and explore its children i.e 8 & 7 here, both 8 & 7 $> \text{threshold}$ so, pruned it. At the end of this, our pruned value is 7 & 8
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 = \text{threshold}$, so, explored its children also i.e 6 & 6, i.e both 6 & 6 = threshold, So, explore one by one,
- The first 6 has two children i.e 6 & 8, having 6 = threshold. So, explore its child also i.e 13 & 7. here both 13 & 7 $> \text{Threshold}$. So, pruned it. next is 8 $> \text{Threshold}$. pruned it, So, pruned value at this stage is 13,7 & 8.
- Explore the second child of 5 i.e 6 = Threshold. So, explore its child i.e 7 & 9. Both are greater than Threshold. So, pruned it

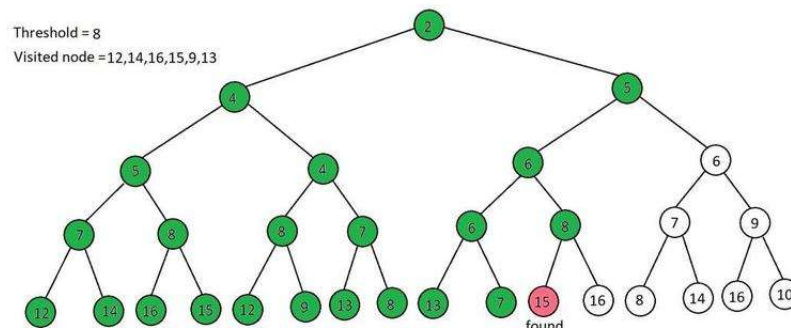
- So, our pruned values are 13,7,8 & 9.

Iteration 5



- In pruned values, the least value is 7, So threshold = 7
- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, the first child is 4, So, set current node = $4 < \text{threshold}$, so, explored its children also i.e 5, 4
- The first child of 4 is 5 i.e $5 < \text{threshold}$ so explore its child also 7&8, Here 7 = threshold. So, explore its children i.e 12 & 14, both $> \text{Threshold}$. So, pruned it. And the second child of 5 is $8 > \text{Threshold}$, So, pruned it. At this stage, our pruned value is 12, 14 & 7.
- Now explore the second child of node 4 i.e 4, so set current node = $4 < \text{threshold}$, and explore its children i.e 8 & 7 here, $8 > \text{threshold}$ so, pruned it. then go to the second child i.e 7 = Threshold, So explore its children i.e 13 & 8. having both $> \text{Threshold}$. So pruned it. At the end of this, our pruned value is 12,14,8 & 13
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 < \text{threshold}$, so, explored its children also i.e 6 & 6, i.e both $6 & 6 < \text{threshold}$, So, explore one by one,
- The first 6 has two children i.e 6 & 8, having $6 < \text{threshold}$. So, explore its child also i.e 13 & 7. here $13 > \text{Threshold}$. So, pruned it. And $7 = \text{Threshold}$. And it hasn't any child. So, the shift to the next sub-child of 6 i.e $8 > \text{threshold}$, So, pruned it. The pruned value at this stage is 12,14,8 & 13
- Explore the second child of 5 i.e $6 < \text{Threshold}$. So, explore its child i.e 7 & 9. Here 7 = Threshold, So, explore its children i.e 8 & 14, Both are greater than Threshold. So, pruned it, Now the sub child of 6 is $9 > \text{Threshold}$, So, pruned it.
- So, our pruned values are 12,14,8,13 & 9.

Iteration 6



Iteration 6

- In pruned values, the least value is 8, So threshold = 8

- current node = root node = 2 and $2 < \text{threshold}$, So explore its children. i.e two children explore one by one
- So, the first child is 4, So, set current node = $4 < \text{threshold}$, so, explored its children also i.e 5, 4
- The first child of 4 is 5 i.e $5 < \text{threshold}$ so explore its child also 7&8, Here $7 < \text{threshold}$. So, explore its children i.e 12 & 14, both $> \text{Threshold}$. So, pruned it. And the second child of 5 is 8 = Threshold, So, So, explore its children i.e 16 & 15, both $> \text{Threshold}$. So, pruned it. At this stage, our pruned value is 12, 14, 16 & 15.
- Now explore the second child of node 4 i.e 4, so set current node = $4 < \text{threshold}$, and explore its children i.e 8 & 7 here, 8 = Threshold, So, So, explore its children i.e 12 & 9, both $> \text{Threshold}$. So, pruned it. then go to the second child i.e $7 < \text{Threshold}$, So explore its children i.e 13 & 8. having $13 > \text{Threshold}$. So pruned it. and 8 = Threshold and it hasn't any child. At the end of this, our pruned values are 12, 14, 16, 15, and 13.
- Similarly, Explore the second child of root node 2 i.e 5 as the current node, i.e $5 < \text{threshold}$, so, explored its children also i.e 6 & 6, i.e both $6 & 6 < \text{threshold}$, So, explore one by one,
- The first 6 has two children i.e 6 & 8, having $6 < \text{threshold}$. So, explore its child also i.e 13 & 7. here $13 > \text{Threshold}$. So, pruned it. And $7 < \text{Threshold}$. And it hasn't any child. So, the shift to the next sub-child of 6 i.e 8 = threshold. So, explored its children also i.e 15 & 16, Here **15 = Goal Node**. So, stop this iteration. Now no need to explore more.
- The goal path is **2→5→6→8→15**

Real-World Applications of IDA*

1. The 15-Puzzle Problem

The 15-puzzle problem is a classic example of a **sliding puzzle game**. It consists of a 4×4 grid of numbered tiles with one tile missing. The aim is to rearrange the tiles to form a specific goal configuration. The state space of the puzzle can be represented as a tree where each node represents a configuration of the puzzle and each edge represents a legal move. IDA* can be used to find the **shortest sequence of moves** to reach the goal state from the initial state.

2. The 8-Queens Problem

The 8-Queens problem is a classic example of the **n-Queens problem** where n queens have been placed on an $n \times n$ chessboard such that no two queens attack each other. The state space of the problem can be represented as a tree where each node represents a configuration of the chessboard and each edge represents the placement of a queen. IDA* can be used to find the **minimum number of queens** that need to be moved to reach the goal state.

In both of these examples, IDA* is used to find the optimal solution by using a combination of depth-first search and a heuristic function to limit the search space. The algorithm incrementally increases the depth bound, allowing it to find the solution without exploring the entire state space, which would be infeasible for larger problems.

Advantages and Disadvantages of Iterative Deepening A* algorithm (IDA*)

Advantages

1. **Optimal Pathfinding:** IDA* guarantees finding the optimal path, as it never overestimates the cost to the goal.
2. **Memory Efficient:** It uses **limited memory** compared to A* by applying depth-first search techniques.

3. **Admissible Heuristic:** The heuristic function ensures that the algorithm remains admissible, thus optimizing performance.
4. **Efficient with Large State Spaces:** IDA* handles large graphs efficiently by pruning unnecessary nodes.

Disadvantages

1. **Repeated Node Exploration:** The algorithm does not store visited nodes, leading to repeated exploration.
2. **Slower than A*:** IDA* can be slower than algorithms like A* due to the repeated exploration of nodes.
3. **Higher Computational Cost:** It may take longer and consume more processing power compared to other algorithms like A* or breadth-first search.

It's important to note that IDA* is not suitable for all types of problems, and the choice of algorithm will depend on the specific characteristics of the problem you're trying to solve.

IDA* vs Iterative Deepening Depth-First Search (IDDFS)

The table given below highlights the differences between IDA* and Iterative Deepening Depth-First Search (IDDFS):

Criteria	Iterative Deepening Depth-First Search (IDDFS)	<i>Iterative Deepening A (IDA)**</i>
Systematic	Systematic exploration of the search space.	Not systematic; may revisit nodes due to threshold updates.
Optimality	Guarantees optimal solution in unweighted graphs.	Optimal, but only expands nodes where $f\text{-score} \leq \text{threshold}$.
Node Expansion	Never expands the same node twice.	May expand the same node multiple times if $f\text{-score} < \text{threshold}$.
Handling Infinite Search Traversal	Not suited for infinite search traversal.	Better suited for infinite search traversal compared to IDDFS.

Applications of IDA* in Artificial intelligence

IDA* is particularly suited for solving complex AI problems where memory is limited. Here are a few areas where it is commonly applied:

1. **Pathfinding in Games:** IDA* is frequently used in video games where memory is a constraint, and optimal paths need to be found efficiently, such as in real-time strategy games.
2. **Robot Navigation:** In robotics, IDA* is applied to efficiently navigate through complex environments without requiring excessive memory.
3. **Puzzle Solving:** Classic AI problems like the 8-puzzle or 15-puzzle can be solved using IDA*, where the search space is large and memory constraints are significant.

4. **Artificial General Intelligence (AGI):** In research areas exploring [AGI](#), where problem-solving often involves navigating vast search spaces, IDA* can be used to manage memory while finding optimal solutions.

4.8 Recursive Best First Search (RBFS)

It is simple recursive algorithm that resembles the operation of standard best first search but uses only linear space. It is similar to recursive DFS and differs from Recursive DFS as follows, It keeps track of the f value of the best alternative path available from any ancestor of the current node. Instead of continuing indefinitely down the current path.

The idea of recursive best first search is to simulate A* search with $O(bd)$ memory, where b is the branching factor and d is the solution depth. It is a memory bound, simple recursive algorithm that works like a standard best first search but only takes up linear space.

There are some things that make it different from recursive DFS. It keeps track of f , the value of the best alternative path that can be found from any ancestor of the current node, instead of continuing indefinitely down the current path.

RBFS mimic the operation of standard Best-First search algorithm. IBFS keep track of the f -value of the best alternative path available from any ancestor of the current node. If the current node exceeds the limit, the recursion unwinds back to the alternative path. As the recursion unwinds, RBFS replaces the f -value of each node along the path with the best f -value of its children. In this way, RBFS remembers the f -value of the best leaf in the forgotten subtree and can therefore decide whether it's worth re-expanding the subtree at some later time.

RBFS is somewhat most efficient than IDA*, but still suffers from excessive node regeneration. A* and RBFS are optimal algorithms if heuristic function $h(n)$ is admissible.

Algorithm

```
{
  function RECURSIVE_BEST_FIRST_SEARCH (Problem)
    returns a solution, or failure
    RBFS (problem, MAKE_NODE (INITIAL_STATE problem), ∞)
  function RBFS (Problem, Node, f-limit) returns a solution or failure and a new f-cost limit
    if GOAL_TEST [Problem] [State] then return node
    Successors ← EXPAND (node, problem)
    if successors is empty then return failure, ∞
    for each S in successors do
       $f(s) \leftarrow \max (g(s) + h(s), f[node])$ 
    repeat
      best ← the lowest f value node in successors
      if [best] > f-limit then return failure, f[best]
      alternative ← the second lowest f-value among successors
      result, f[best] ← BFS (Problem, best, min (f-limit, alternative))
      if result ≠ failure then return result
}
```

Advantages

- More efficient than IDA*
- It is an optimal algorithm if $h(n)$ is admissible
- Space complexity is $O(bd)$.

Disadvantages

- It suffers from excessive node regeneration.
- Its time complexity is difficult to characterize because it depends on the accuracy of $h(n)$ and how often the best path changes as the nodes are expanded.

Advantages

- More efficient than IDA* and still optimal.
- Best-first Search based on next best f -contour; fewer regeneration of nodes.
- Exploit results of search at a specific f -contour by saving next f -contour associated with a node whose successors have been explored.
- Like IDA* still suffers from excessive node regeneration.
- IDA* and RBFS not good for graphs.
- Can't check for repeated states other than those on current path.
- Both are hard to characterize in terms of expected time complexity.

4.10 Pruning the closed list

Pruning the closed list in AI search algorithms, like A*, involves discarding paths that have already been explored to reduce redundant computation and improve efficiency. This is done by maintaining a closed list (explored set) of nodes and discarding paths that lead to nodes already in the closed list. Here's a breakdown with an example:

What is the Closed List?

- The closed list (or explored set) is a data structure (usually a set or hash table) that stores nodes that have already been expanded and evaluated during the search process.
- It helps avoid revisiting nodes that have already been explored, preventing infinite loops and redundant calculations.

How Pruning Works**1. Initialization:**

Start with an empty closed list.

2. Path Selection:

When a path $\langle n_0, \dots, n_k \rangle$ is selected for expansion, check if the last node n_k is already in the closed list.

3. Pruning:

- If n_k is in the closed list, the path can be discarded because it has already been explored.
- Otherwise, n_k is added to the closed list, and the algorithm proceeds as before.

4. Repeat:

Continue this process until a solution is found or the search space is exhausted.

Example: A Algorithm*

Imagine you're using the A* algorithm to find the shortest path between two points on a map.

- **Open List:** Contains nodes that are currently being considered for expansion (nodes that are "on the fringe" of the search).
- **Closed List:** Contains nodes that have already been expanded and evaluated.
- **Pruning in A*:**
 - If you find a path to a particular node and that node is already in the closed list, you can discard this path because you've already found a path to that node from a previous expansion.
 - If the new path to the node has a higher cost than the path already in the closed list, you can also discard the new path.

Benefits of Pruning

- **Reduced Search Space:**

By avoiding redundant exploration, pruning significantly reduces the size of the search space, leading to faster search times.

- **Improved Efficiency:**

Pruning helps algorithms like A* to find optimal solutions more efficiently.

- **Avoids Infinite Loops:**

The closed list prevents algorithms from getting stuck in cycles or revisiting the same nodes repeatedly.

Note: Pruning the closed list is a common technique in AI search algorithms that helps to improve their performance and efficiency.

4.9 Divide and Conquer Beam Stack Search:

Divide and conquer algorithms are a fundamental technique in [data structures and algorithms](#), used to solve complex problems by breaking them down into smaller, more manageable subproblems.

These algorithms are widely used in computer science, with popular examples including [Merge Sort](#), [Quick Sort](#), and Binary Search. Let's learn how divide and conquer algorithm works, its key benefits, and its practical applications, helping you understand why it's such an important strategy in [algorithm](#) design.

What is Divide and Conquer Algorithm?

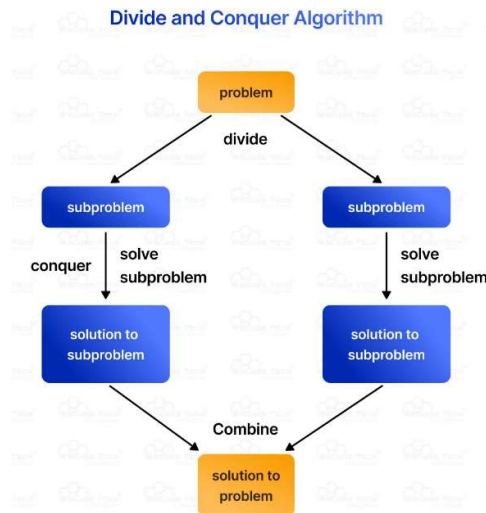
A divide and conquer algorithm is a method used in computer science to solve big problems by breaking them down into smaller, more manageable parts. The idea is to "divide" the problem into smaller pieces, "conquer" each piece by solving it, and then "combine" the solutions to get the final answer.

For Example:

Imagine you have a big jigsaw puzzle with 1,000 pieces. Instead of trying to solve the whole puzzle at once, you could divide it into smaller sections, like the corners, edges, and middle. You solve each section separately, and then you combine them to complete the entire puzzle. This approach is similar to how a divide and conquer algorithm works.

How Divide and Conquer Algorithm Works?

The steps in a divide and conquer method are straightforward and can be broken down into three main parts: Divide, Conquer, and Combine.



1. Divide

The first step is to break down the larger problem into smaller, more manageable subproblems. This "dividing" process continues until the subproblems are simple enough to be solved directly, often when they reach a base case, like a single element or a very small dataset.

2. Conquer

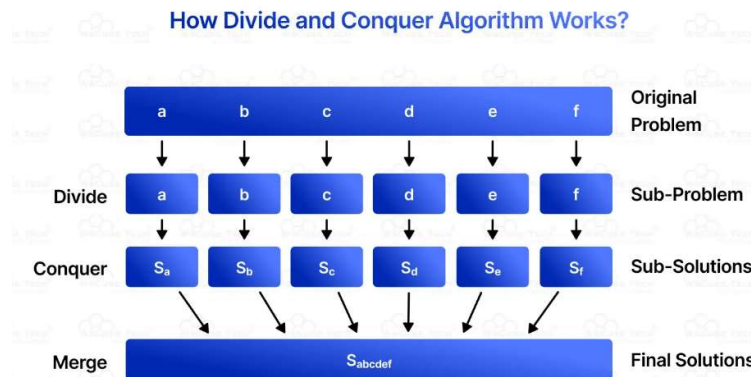
After dividing the problem, the next step is to "conquer" each of the smaller subproblems. This usually involves solving each subproblem individually.

Depending on the algorithm, this step might involve recursion, where the subproblems are solved using the same divide and conquer approach, or it might be done through simple, direct computation.

3. Combine

Finally, once all the subproblems have been solved, the last step is to "combine" the solutions of these subproblems to form the solution to the original, larger problem.

This step merges the solutions in a way that completes the task, whether it's sorting a list, multiplying matrices, or finding the maximum value in a dataset.



Divide and Conquer Algorithm Examples

Below are some examples of divide and conquer algorithm:

1. Merge Sort

For an array [38, 27, 43, 3, 9, 82, 10]:

Step 1 (Divide):

- Split into [38, 27, 43] and [3, 9, 82, 10].

Step 2 (Divide further):

- [38, 27, 43] is split into [38] and [27, 43]

- [3, 9, 82, 10] is split into [3, 9] and [82, 10]

Step 3 (Base case):

[38], [27], [43], [3], [9], [82], [10] are all single-element arrays.

Step 4 (Conquer and Combine):

- Merge [27] and [43] to get [27, 43].
- Then, merge [38] with [27, 43] to get [27, 38, 43].
- Similarly, merge [3, 9] and [10, 82],
- finally merge all together to get [3, 9, 10, 27, 38, 43, 82].

2. Quick Sort

For an array [10, 7, 8, 9, 1, 5] with 5 as the pivot:

Step 1 (Divide):

Partition into [1] (less than 5) and [10, 7, 8, 9] (greater than 5).

Step 2 (Conquer):

Apply Quick Sort recursively to [1] (which is already sorted) and [10, 7, 8, 9].

Step 3 (Further Division):

Choose a new pivot from [10, 7, 8, 9], say 9, and partition into [7, 8] (less than 9) and [10] (greater than 9).

Step 4 (Combine):

Since sub-arrays [1], [5], [7, 8], [9], and [10] are now sorted, the final array is [1, 5, 7, 8, 9, 10].

Time Complexity for Divide and Conquer Algorithms

<u>Algorithm</u>	<u>Time Complexity</u>
Merge Sort	$O(n \log n)$
Quick Sort	• $O(n \log n)$ (average case) • $O(n^2)$ (worst case)
Binary Search	$O(\log n)$
Strassen's Matrix Multiplication	$O(n^{2.81})$
Karatsuba Algorithm (Multiplication of large numbers)	$O(n^{1.585})$
Closest Pair of Points	$O(n \log n)$
Convex Hull (Divide and Conquer)	$O(n \log n)$

Advantages of Divide and Conquer Algorithms

- **Efficiency:** Many divide and conquer algorithms, like Merge Sort and Quick Sort, are highly efficient and have better time complexity compared to simpler algorithms.
- **Parallelism:** The independent subproblems can often be solved in parallel, making these algorithms well-suited for parallel and distributed computing environments.
- **Modularity:** Divide and conquer algorithms break down complex problems into smaller, more manageable subproblems, making the code easier to understand, implement, and debug.

- **Scalability:** These algorithms perform well on large datasets, which makes them scalable for real-world applications involving big data.
- **Reusability:** The same divide and conquer approach can be applied to different problems, making it a versatile strategy in algorithm design.

Disadvantages of Divide and Conquer Algorithms

- **Overhead:** Recursive function calls and managing the division of the problem can introduce additional overhead, which may lead to inefficiency for small or simple problems.
- **Complexity in Combining Solutions:** In some cases, combining the solutions of subproblems can be complex and may negate the benefits of dividing the problem.
- **Not Always the Best Choice:** For problems that are inherently simple, a divide and conquer approach may be overkill and less efficient than a straightforward algorithm.
- **Space Complexity:** Some divide and conquer algorithms, like Merge Sort, require additional space for combining solutions, which can be a drawback in memory-constrained environments.

Applications of Divide and Conquer Algorithms

1. Sorting Algorithms

- **Merge Sort:** Used in various applications where stable sorting is required, such as in sorting large datasets, merge operations in databases, and external sorting.
- **Quick Sort:** Commonly used in sorting operations where speed is crucial, like in-memory sorting, system libraries, and commercial databases.

2. Searching Algorithms

- **Binary Search:** Widely used in searching for elements in sorted arrays, searching in databases, and implementing efficient search operations in software applications.

3. Matrix Multiplication

- **Strassen's Algorithm:** Applied in numerical linear algebra, scientific computing, and in systems where fast matrix multiplication is needed, such as in graphics processing and simulations.

4. Multiplication of Large Numbers

- **Karatsuba Algorithm:** Used in cryptography, large integer arithmetic, and computer algebra systems for fast multiplication of large numbers.

4.12 Beam-stack search:

Beam-stack search is an AI search algorithm that combines chronological backtracking (depth-first search) with beam search, finding good solutions quickly and then backtracking to find improved solutions until convergence to an optimal solution.

Here's a more detailed explanation:

- **What it is:**

Beam-stack search is an anytime algorithm, meaning it can provide a solution at any point in time, and it continues to search for better solutions.

- **How it works:**

- It uses a "beam stack" data structure to integrate systematic backtracking with beam search.
- It starts by finding a good but potentially suboptimal solution quickly, similar to beam search.

- Then, it backtracks and continues searching for improved solutions until it converges to an optimal solution.
- **Advantages:**
 - It finds good solutions quickly.
 - It can continue searching for better solutions until convergence to an optimal solution.
- **Disadvantages:**
 - It is an anytime algorithm, so it may not find the optimal solution immediately.
- **Relationship to other algorithms:**
 - It's similar to depth-first beam search.
 - It's an alternative to greedy search, which only samples the most likely token at every step.
 - It can be viewed as a generalization of breadth-first search (BFS)

Thus beam-stack search is an anytime algorithm that finds an initial solution relatively quickly, and continues to search for improved solutions until convergence to an optimal solution. (Note that each time beam-stack search finds an improved solution, it updates its upper bound.)